

# BHP/1: Binary HyperText Protocol, version 1

BHP/1 carries HTTP-style GET/HEAD requests and their responses over one persistent TCP connection, in binary frames whose length is known before a single payload byte is read. All integers are unsigned and **big-endian** (network order). Text is UTF-8.

## 1. Connection

The client connects and first sends a 4-byte **preface**: 42 48 50 01 ("BHP" + version 1). A server that receives anything else **MUST** send GOAWAY PROTOCOL\_ERROR and close. After the preface, both sides exchange only frames. The connection stays open after each response. Either side may close it at any time, and **SHOULD** send GOAWAY first (§6). A server that receives GOAWAY closes the connection.

*Why a preface?* The version is stated once per connection, not once per frame, and it fails fast if a text client like curl connects to us by mistake.

## 2. Frame header (8 bytes)

|       |             |   |   |      |       |                |   |   |         |
|-------|-------------|---|---|------|-------|----------------|---|---|---------|
| byte: | 0           | 1 | 2 | 3    | 4     | 5              | 6 | 7 | 8 ...   |
|       | Length (24) |   |   | Type | Flags | Stream ID (24) |   |   | Payload |

| Field     | Bits | Why this width                                                                                                                                                                                                                                                                                                                                                                      |
|-----------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Length    | 24   | The number of payload bytes that follow. 16 bits (64 KiB) would cap any future version, and 32 bits costs a byte on every frame to describe sizes no single frame should carry. v1 caps payloads at <b>16,384 bytes</b> , which bounds a receiver's per-frame memory. The extra width is room for v2 to raise that cap without changing the header.                                 |
| Type      | 8    | 3 types are used and 253 are free for future versions.                                                                                                                                                                                                                                                                                                                              |
| Flags     | 8    | One bit is used (0x01 END_STREAM, meaning this is the stream's last frame). Unknown bits <b>MUST</b> be ignored.                                                                                                                                                                                                                                                                    |
| Stream ID | 24   | Pairs each response with its request. 16 bits (65,536 requests) can run out within minutes on a busy connection. HTTP/2 uses 31 bits, but that pushes the header to 9 bytes. 24 bits gives 16.7 million requests per connection and keeps the header at <b>8 bytes, two aligned 32-bit words</b> that are easy to read in a hexdump. A client that runs out opens a new connection. |

**Frame types.** 0x01 HEADERS, 0x02 DATA, 0x03 GOAWAY. All others are reserved.

**Extensibility rule.** A receiver that meets a frame type it does not know **MUST read and discard** Length bytes and carry on. This is always possible because Length sits at the same place in every frame. It is how a v2 can add frame types without breaking v1 peers.

## 3. Streams

Stream 0 is the connection itself and is used only by GOAWAY. The client gives each request a new ID: 1, 2, 3, ..., **strictly increasing**. A HEADERS frame on stream 0, or with an ID that is not higher than the previous one, is a connection error. The server answers on the same stream ID, **in the order the requests arrived**, so the client MAY pipeline (send several requests before reading). A response is **exactly one HEADERS frame, then zero or more DATA frames**. The stream ends at the first frame carrying END\_STREAM.

## 4. HEADERS payload

|                              |                |                            |                  |        |
|------------------------------|----------------|----------------------------|------------------|--------|
| Request (client -> server):  | Method (1)     | Path length (2)            | Path             | Field* |
| Response (server -> client): | Status (2)     | Field*                     |                  |        |
| Field:                       | Name index (1) | [ Name length (1)   Name ] | Value length (2) | Value  |

- **Method:** 0x01 GET, 0x02 HEAD. Any other code is well-formed, and the server answers 405. A request HEADERS frame **SHOULD** set END\_STREAM, because v1 requests have no body.
- **Path** MUST start with / and MUST NOT contain NUL. It is raw UTF-8, with no percent-encoding. A ? starts a query, which bserve ignores. A path naming a directory serves that directory's index.html.
- **Status** is 16 bits. One byte cannot hold 404, and 10 bits would save nothing once rounded to whole bytes.
- **Fields run until the end of the payload.** There is no count: the frame Length already marks the end, and a second number could only disagree with it.
- **Name index** 1..10 refers to the **static table** below. Index 0 means a *literal* name follows: 1 length byte, then a lowercase name of 1–255 bytes. Indexes 11..255 are reserved, and receiving one makes the message malformed. Name lengths are 1 byte because names are short. Values get 2 bytes, up to 65,535.

| # | Name       | # | Name            | # | Name           | # | Name          | #  | Name   |
|---|------------|---|-----------------|---|----------------|---|---------------|----|--------|
| 1 | host       | 3 | accept          | 5 | content-type   | 7 | last-modified | 9  | server |
| 2 | user-agent | 4 | accept-encoding | 6 | content-length | 8 | etag          | 10 | date   |

These are the ten names BHP/1 peers actually send. Anything else, for example allow on a 405, goes as a literal. This uses the first two mechanisms of HPACK, HTTP/2's header compression: a static table, and length-prefixed literals.

## 5. DATA payload

DATA carries body bytes, split into frames of at most 16,384 bytes, with **END\_STREAM on the last one**. A response with no body (HEAD, an empty file) sets END\_STREAM on its HEADERS frame and sends no DATA. A server MUST discard DATA frames it receives, since v1 requests have no body. content-length, when present, MUST equal the total DATA length, except in a response to HEAD, where it gives the length GET would have sent.

## 6. GOAWAY payload and errors

| GOAWAY (stream 0):    Last stream ID (3)   Error code (1)   Debug text (UTF-8, rest of payload) |                 |                                                       |
|-------------------------------------------------------------------------------------------------|-----------------|-------------------------------------------------------|
| Code                                                                                            | Name            | Sent when                                             |
| 0x00                                                                                            | NO_ERROR        | Graceful close, e.g. the server's idle timeout        |
| 0x01                                                                                            | PROTOCOL_ERROR  | Bad preface or version; stream ID 0 or not increasing |
| 0x02                                                                                            | FRAME_TOO_LARGE | Length > 16,384                                       |
| 0x03                                                                                            | TIMEOUT         | A frame was started but not finished in time          |

There are **two levels of error**, decided by one question: *do we still know where the next frame starts?*

- **Stream error.** Yes, because the frame was framed correctly but its payload is wrong: a truncated field, a reserved index, a bad path. The server answers that stream with **status 400** and the connection carries on. Also stream-level: 404 (no such file, or a path outside the document root), 403 (unreadable), 405 (unsupported method), 500 (server bug). If something fails *after* a response has started, the server closes the connection, and the client sees a stream without END\_STREAM.
- **Connection error.** No, or the peer can no longer be trusted. Send GOAWAY with the code from the table, then close. *Last stream ID* tells the client which requests were processed and which are safe to retry.

## 7. Limits and timeouts (server)

| Limit                          | Value | Defence                                                                                                              |
|--------------------------------|-------|----------------------------------------------------------------------------------------------------------------------|
| Idle time between requests     | 60 s  | A person typing requests by hand is not cut off. On expiry the server sends GOAWAY NO_ERROR.                         |
| Time to finish a started frame | 30 s  | One total deadline, so a client sending a byte every few seconds cannot hold the connection open forever.            |
| Concurrent connections         | 100   | Bounds the threads and file descriptors used by idle connections. Connections over the limit are closed immediately. |

## 8. Example

GET /a with one header, host: h, on stream 1:

|                                                                  |
|------------------------------------------------------------------|
| 00 00 09 01 01 00 00 01   01 00 02 2f 61   01 00 01 68           |
| length=9 HDRS END stream=1   GET len=2 "/a"   #1(host) len=1 "h" |

A complete, byte-by-byte annotated request and response, captured from the real programs, is in docs/HEXDUMP.md.